

Course: Fundamentals of Generative AI and Large Language Models: Theory and Practice

Week 1: Fundamentals of Deep Learning for Generative AI

**AE → VAE → GAN → Diffusion → Sequence models → LLMs → RAG →
Multimodal AI and Ethics**

Week 1 Outline

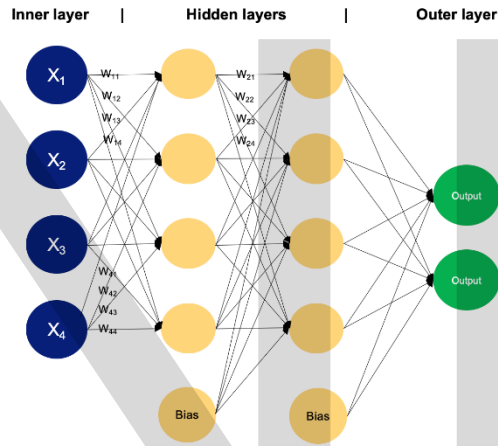
- Understanding the shift from discriminative learning to generative learning
- What is generative modelling
- Types of generative modelling:
 - ✓ Unsupervised generation
 - ✓ Joint modelling
 - ✓ Conditional generation
- Probability distributions in generative models:
 - ✓ Gaussian
 - ✓ Bernoulli
 - ✓ Categorical
- Applications of Generative AI
- Basics of Deep Learning: Activation functions, Loss, Optimizers in Neural Networks
- Convolutional Neural Networks:
 - ✓ Key components of CNN

Hands-on Exercise 1: Build a CNN for a real-world dataset.

Hands-on Exercise 2: Building an ensemble model.

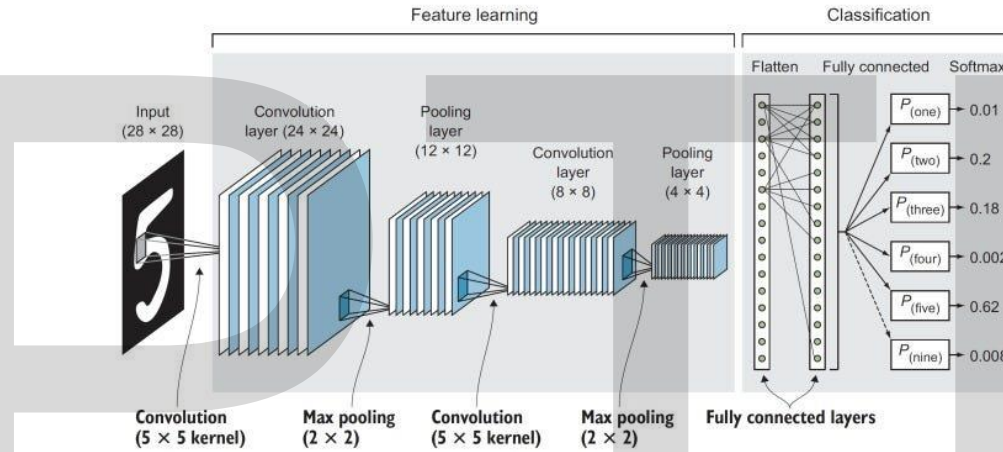
Where we are coming from (Discriminative mindset)

So far, we have built models that learn a mapping from inputs to outputs:



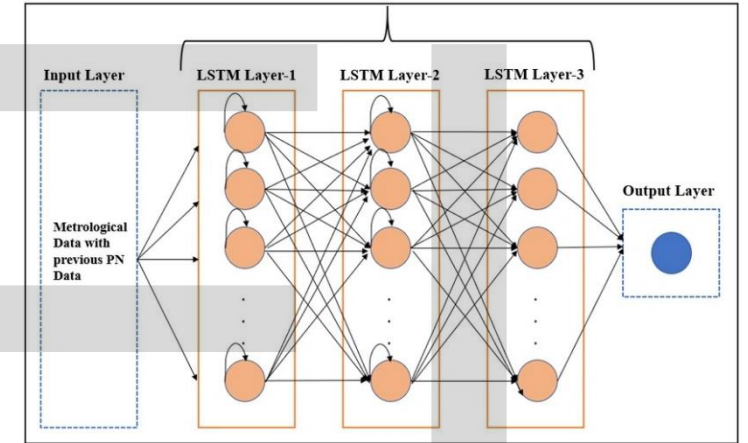
MLP / Feedforward networks →

mainly for **classification and regression** on tabular or vector features.



CNNs →

for **image understanding** tasks (image → class).



RNN / LSTM / GRU →

for **sequential/time-series modeling** (sequence → prediction).

Given a labelled dataset,
 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

Where, x = input (image, text embedding, features, time-series window, etc.) and y = label/target (class label or numeric value)



Discriminative models learn to **discriminate between classes**. The model learns features that help separate **one output from another** using examples where the correct answer is already known. They do not mainly learn how the data was generated. They learn:

Input $x \rightarrow$ Output y

$$P(y | x)$$

or more specifically:

Given the input x , what is the most likely label or output y ?

$x = \text{image}$

The model may output:

$$P(\text{cat} | x) = 0.85$$

$$P(\text{dog} | x) = 0.10$$

$$P(\text{horse} | x) = 0.05$$

So, the model is learning which class the input belongs to.

Why Move Beyond Discriminative Learning?

Earlier Deep Learning focus: Prediction and Classification

Traditional Deep Learning models mainly focused on:

$$x \rightarrow y$$

Given an input x , predict the output y .

Examples:

image $\rightarrow$ class label

patient features $\rightarrow$ disease / no disease

time-series sequence $\rightarrow$ future value

These models are powerful, but they mainly answer:

What is next?

or

Which class it belongs to?

But, **Modern Deep Learning is not only about classifying or predicting.**

It is increasingly about:

Learning the data distribution

So that models can:

Generate new, realistic, and statistically similar data

Examples:

text prompt $\rightarrow$ new image

text prompt $\rightarrow$ new paragraph

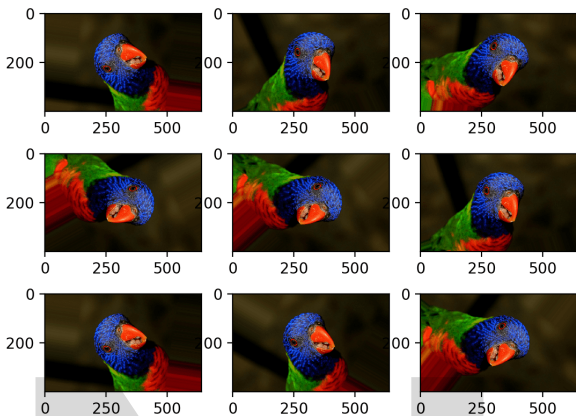
damaged image $\rightarrow$ completed image

noisy data $\rightarrow$ clean reconstructed data

existing samples $\rightarrow$ new synthetic samples

These models try to answer:

What new data can be created?



If the need is just to generate the data,
then the existing methods such as

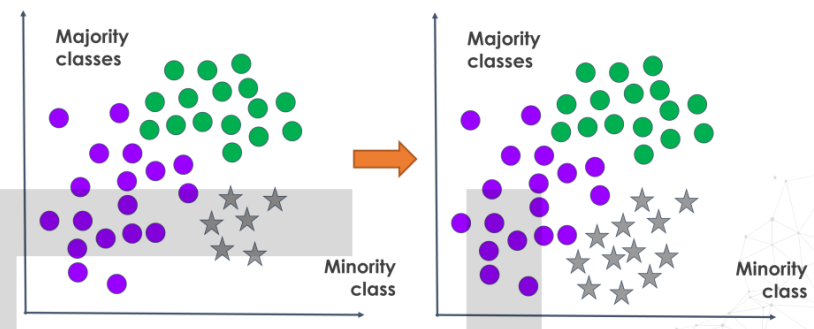
Common fixes

Data augmentation: Data augmentation creates variations of existing samples.

For images, examples include:
flip, rotation, zoom, crop, noise

But augmentation usually creates **modified versions of existing data**.

It does not fully learn:
the *true underlying data distribution*



Oversampling: Oversampling increases the number of minority-class samples.

For example, it may duplicate or interpolate minority-class examples.

But oversampling mainly improves the **count** of samples.

It may not always create rich and realistic new variations.

Generative modeling aims to learn the structure and distribution of data, so that **new, realistic, and diverse samples** can be generated.

What is Generative Modelling?

A generative model learns the **probability distribution of data**, so that it can generate new samples similar to the training data.

Discriminative model:

$$P(y | x)$$

Generative model:

$$P(x) \text{ or } P(x, y)$$

1. Unsupervised Generation

Here, the model learns only from data samples.

$$x_1, x_2, x_3, \dots, x_n \rightarrow p_{data}(x)$$

There are **no labels**.

The model learns:

$$p_{\theta}(x)$$

After training:

$$p_{\theta}(x) \approx p_{data}(x)$$

The model has learned what real data look like.

Then it can generate a new data:

$$x_{new} \sim p_{\theta}(x)$$

Age (years)	Height (cm)	Weight (kg)
6	115	20
8	125	25
10	135	30
12	145	36
14	155	45
16	165	55
18	170	62
20	172	66
22	174	69
25	175	72

Here, only data is given, without labels.

For example, consider many digit images.

The model learns the general pattern of digit images and then generates new digit-like images.

2. Joint Modelling

Here, the model learns both the data and the label together.

where:

$$p_{\theta}(x, y)$$

$$x = \text{data}$$

$$y = \text{label}$$

For example, it learns both the image and its class label. Once it learns this joint distribution, Bayes' theorem can be used to perform classification.

$$p(y|x) = \frac{p(x|y) p(y)}{p(x)}$$

3. Conditional Generation

Generates data based on a condition:

$$p_{\theta}(x | y) \text{ or } p_{\theta}(x | c)$$

Examples:

“Generate digit 7”

“Generate a smiling face”

“Generate a medical image with tumor”

Output Distribution: What Does the Decoder Generate?

After the decoder or generator produces an output, the type of output decides the probability distribution we assume.

If the output is a continuous real-valued value:

$$x \in \mathbb{R} \text{ or } x \in [0,1]$$

Example: normalized pixel value, sensor value

Model it using a **Gaussian distribution**.

If the output is binary:

$$x \in \{0, 1\}$$

Example: black/white pixel, yes/no output

Model it using a **Bernoulli distribution**.

If the output is a word/token from a vocabulary:

$$x \in \{1, 2, 3, \dots, K\}$$

Example: next word prediction in language models

Model it using a **Categorical distribution**.

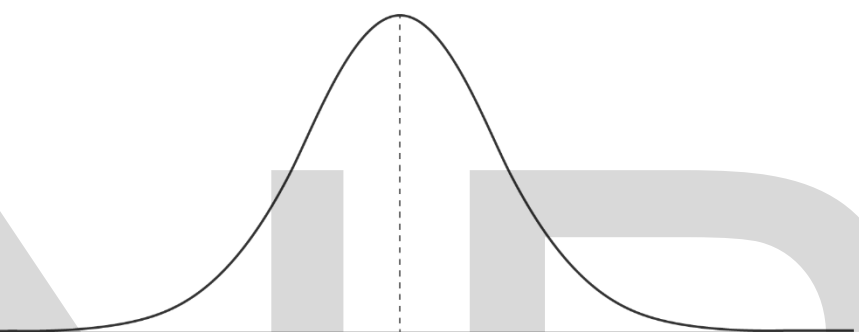
During implementation, we usually do not explicitly write “Gaussian,” “Bernoulli,” or “Categorical.”

Instead, the distribution is implied through:

- 1. Output layer activation function**
- 2. Loss function**

<i>Distribution</i>	<i>Output activation + Loss function</i>
Gaussian	<pre>outputs = Dense(784, activation='linear')(x) model.compile(loss='mse')</pre> Each output value is treated as a continuous value. The model predicts the mean of a Gaussian distribution. linear or sigmoid output + mse loss
Bernoulli	<pre>outputs = Dense(784, activation='sigmoid')(x) model.compile(loss='binary_crossentropy')</pre> when the output data is binary. Each output value is treated as the probability of 1. sigmoid output + binary_crossentropy loss
Categorical	<pre>outputs = Dense(vocab_size, activation='softmax')(x) model.compile(loss='categorical_crossentropy')</pre> The output is a probability distribution over many possible categories. softmax output + categorical_crossentropy loss

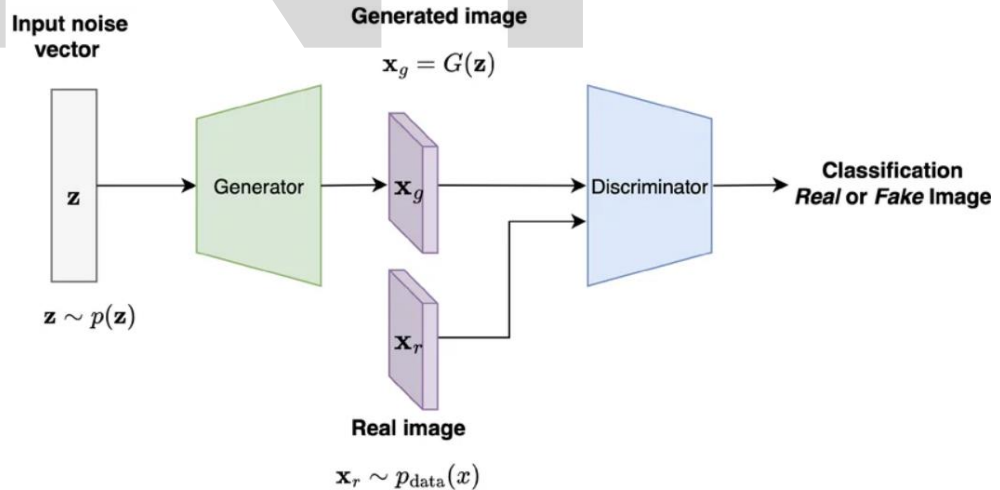
Types of Generative Models



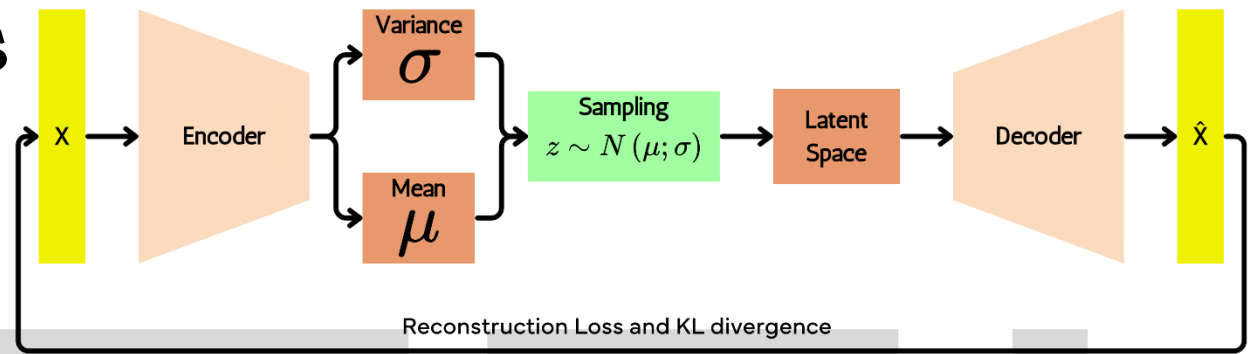
Gaussian Distribution

It is a continuous distribution. It is defined by two parameters:
Mean (μ): Which is center of the curve

Standard Deviation (σ): which determines the spread.



Variational Autoencoder

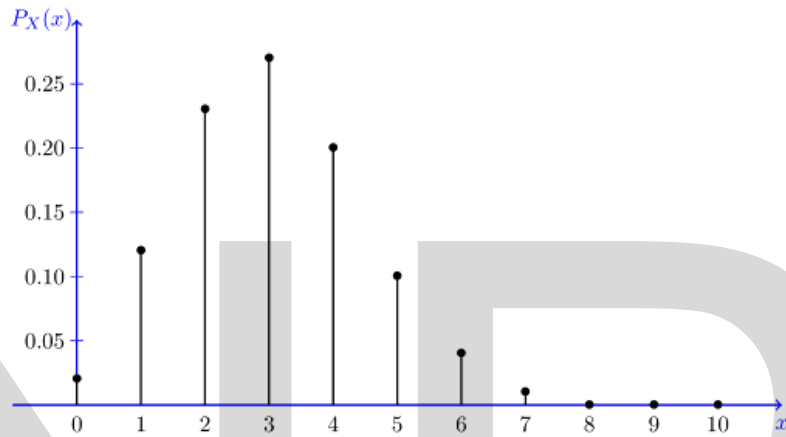


In VAE, the encoder maps input data into a Gaussian distribution in the latent space. New data is then generated by sampling from this distribution.



Diffusion models gradually add Gaussian noise to an image until it becomes nearly pure noise. The model then learns to reverse this process by denoising step by step to generate a new image.

In GANs, the generator starts from a random noise vector z , usually sampled from a simple Gaussian distribution.



Bernoulli Distribution

This is the simplest discrete distribution, with only two possible outcomes: Success(1) or failure (0). It is defined by a single parameter p , the probability of success.

Role in GenAI:

Binary Data: Used when generating binary outputs, such as black-and-white (pixel on/off) images.

Binary classification: Often used in the Discriminator part of GAN to output (0,1) that an image is “real” or “fake”

Categorical Distribution

It represents the probability that a variable falls into one of k distinct categories.

Role in GenAI:

Text generation: When LLMs predicts the “next token”, it calculates a categorical distribution over its entire vocabulary and then samples the most likely next word.

Applications of Generative AI

Text

Text Generation:

- Drafting
- Summarization
- Translation

Common models:

LLMs / Transformers to create human-like, contextually relevant text

Visual

- Image/video generation
- Editing
- Enhancement
- 3D content

Common models:

GANs, VAEs, Diffusion to create, edit, or enhance visual content

Audio

- Speech
- Music
- Sound generation

Common models:

Transformers, GANs for audio, **VAEs** for musical structure

Code

- Code generation

Common models:

LLMs, Transformers, diffusion

Next session

Deep Learning Foundations for Generative AI

- Activation functions
- Loss functions

NLPTEL

NIPTELE

Thank you